

AhamX Bodhi

Where I Becomes Infinite

Level 2: Model Adaptation

Module 2: Generative AI – Core Concepts & Architecture
Prof. Sashikumaar Ganesan · IISc Bengaluru



Session Overview

Session Details

- **Module:** 2 – Core Concepts & Architecture
- **Duration:** 10 minutes
- **Bloom's Level:** B3 (Apply)
- **Difficulty:** L2 (Intermediate)
- **Sequence:** Concept 2 of Module 2

The Central Question

A foundation model is pre-trained on everything. But your task is specific.

How do you bridge that gap?

This session answers that with three distinct adaptation strategies, each with different cost, data, and quality trade-offs.

Learning Objectives



By the End of This Session You Will Be Able To

1. **Define** model adaptation and explain why foundation models require it
2. **Identify** the three primary adaptation strategies: prompting, RAG, and fine-tuning
3. **Compare** each strategy across cost, data requirements, and performance
4. **Apply** a decision framework to select the right adaptation approach for a given scenario
5. **Recognise** production systems that exemplify each adaptation strategy



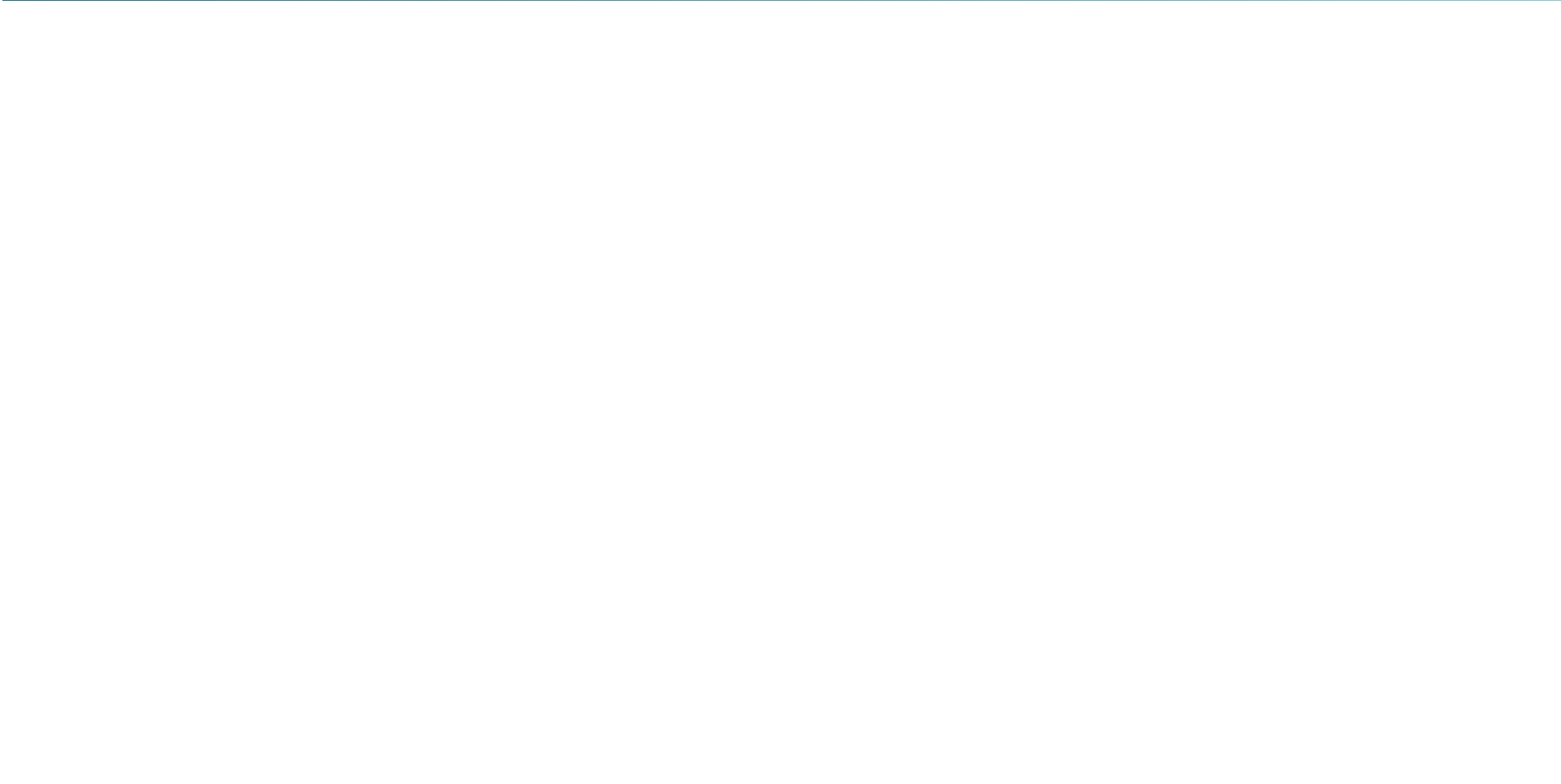
Prerequisites and Context

You Should Already Know

- What a foundation model is and its four properties
- How self-supervised pre-training works
- The landscape of open-weight vs. closed-API models
- The Five-Level GenAI Framework (Level 2 = Adapt)

This Session Unlocks

- Prompt Engineering deep-dive (Module 2)
- RAG Architecture and pipelines (Module 5)
- Fine-tuning: LoRA and QLoRA (Module 7)
- RLHF and instruction tuning (Module 7)
- Model evaluation and benchmarking (Module 8)





The Adaptation Problem

What Pre-Training Gives You

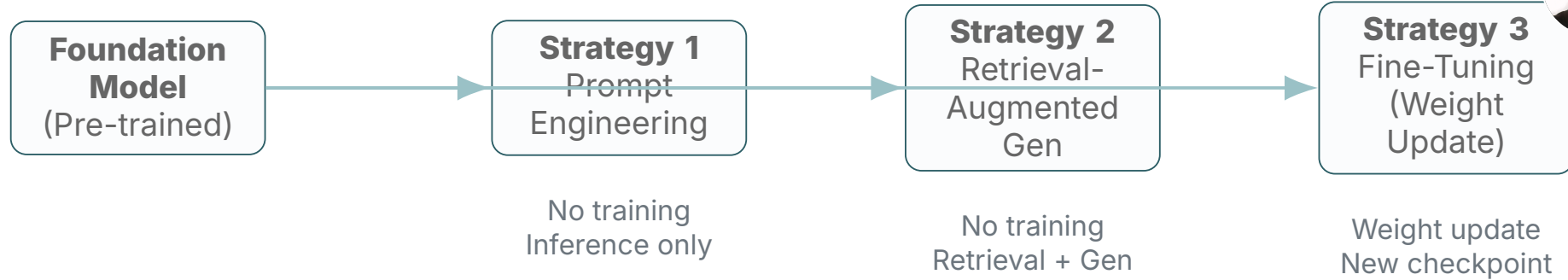
A foundation model pre-trained on the internet knows a great deal about language, facts, and reasoning. But it has **no knowledge of your organisation's documents, no alignment to your tone or format requirements, and no instructions** about which tasks to prioritise or refuse. It is a powerful but undirected engine.

What Adaptation Adds

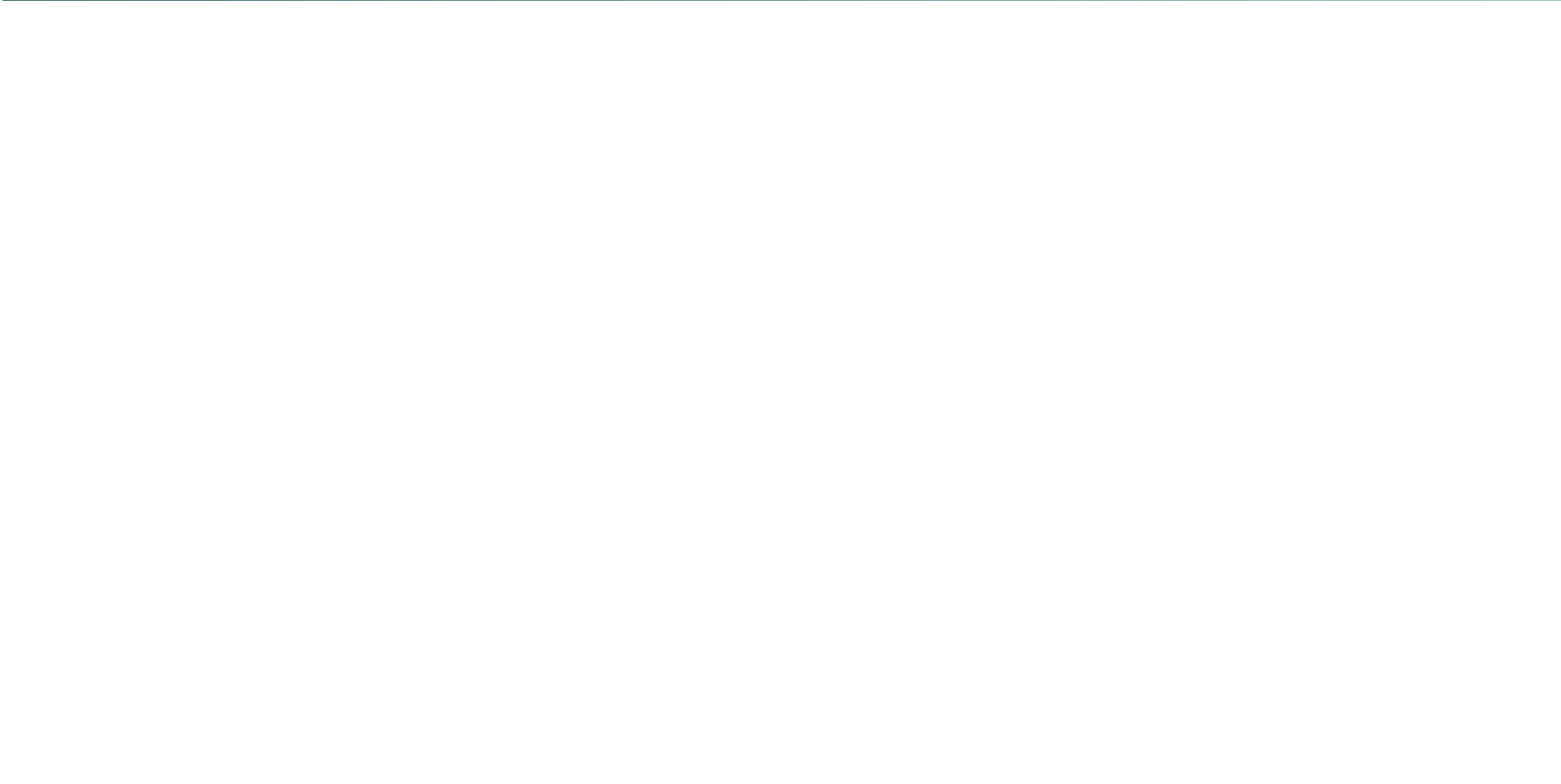
- **Task specificity:** Direct the model to a narrow, well-defined purpose
- **Knowledge grounding:** Inject domain-specific facts the model was not trained on
- **Behaviour alignment:** Instil tone, refusal rules, format constraints
- **Performance improvement:** Exceed general-purpose accuracy on specialised benchmarks



Three Strategies for Adapting Foundation Models



Each strategy sits at a different point on the **cost-effort-performance** spectrum. Choosing correctly is one of the most impactful decisions an AI engineer makes.





Prompt Engineering: Definition and Mechanics

What Is Prompt Engineering?

Prompt engineering is the practice of designing, structuring, and optimising the input text (the prompt) given to a foundation model to elicit the desired output — **without modifying any model weights**. The model's behaviour is steered entirely through the context provided at inference time.

Core Prompting Techniques

- **Zero-shot:** Instruction only, no examples – “Classify this review as positive or negative”
- **Few-shot:** 2–5 input-output examples prepended to the instruction
- **Chain-of-Thought (CoT):** Instruct the model to reason step-by-step before answering
- **System prompts:** High-level persona, constraints, and format rules set once per session
- **Role prompting:** “You are a senior tax attorney. Advise on...”



Prompt Engineering: Strengths and Limitations

Strengths

- **Zero setup cost:** No data, no training, no new model
- **Immediate:** Results in minutes, not days
- **Flexible:** Iterate without infrastructure changes
- **Always the first choice** – exhaust this before moving to RAG or fine-tuning

Limitations

- Context window bounded – cannot inject unlimited knowledge
- Fragile to rephrasing – subtle wording changes shift outputs
- Cannot inject private/real-time knowledge beyond the window
- Does not change underlying model capabilities or knowledge cutoff





Retrieval-Augmented Generation: Definition

What Is RAG?

Retrieval-Augmented Generation (RAG) augments a foundation model's response by first **retrieving** the most relevant passages from an external document store and then **prepending** them to the model's context window as grounding evidence. The model generates a response conditioned on both the query and the retrieved documents. **No weight updates occur.**

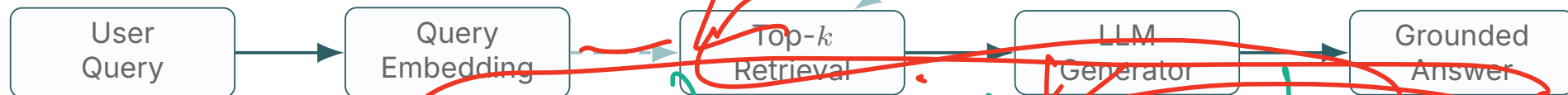
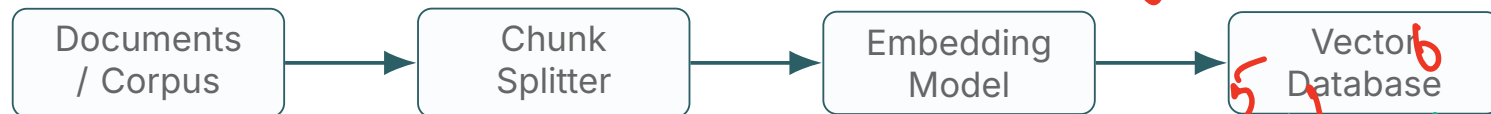
The Four-Step RAG Pipeline

- **Step 1 – Index:** Documents are split into chunks and encoded by an embedding model into dense vectors
- **Step 2 – Store:** Vectors are stored in a vector database (Pinecone, Weaviate, pgvector)
- **Step 3 – Retrieve:** User query is embedded; top- k nearest chunks fetched by cosine similarity
- **Step 4 – Generate:** Retrieved chunks + query are passed to the LLM for grounded generation

RAG Architecture: Index and Query Paths



— Indexing Path —



— Query Path —

The indexing path runs **once** (or on a schedule). The query path runs **at inference time** for every user request – this is what makes RAG capable of serving up-to-date, private knowledge without retraining.

RAG: Strengths and Limitations



$$T_i \cdot V_j \quad i = 1, \dots, N_T$$

$$j = 1, \dots, N_V$$

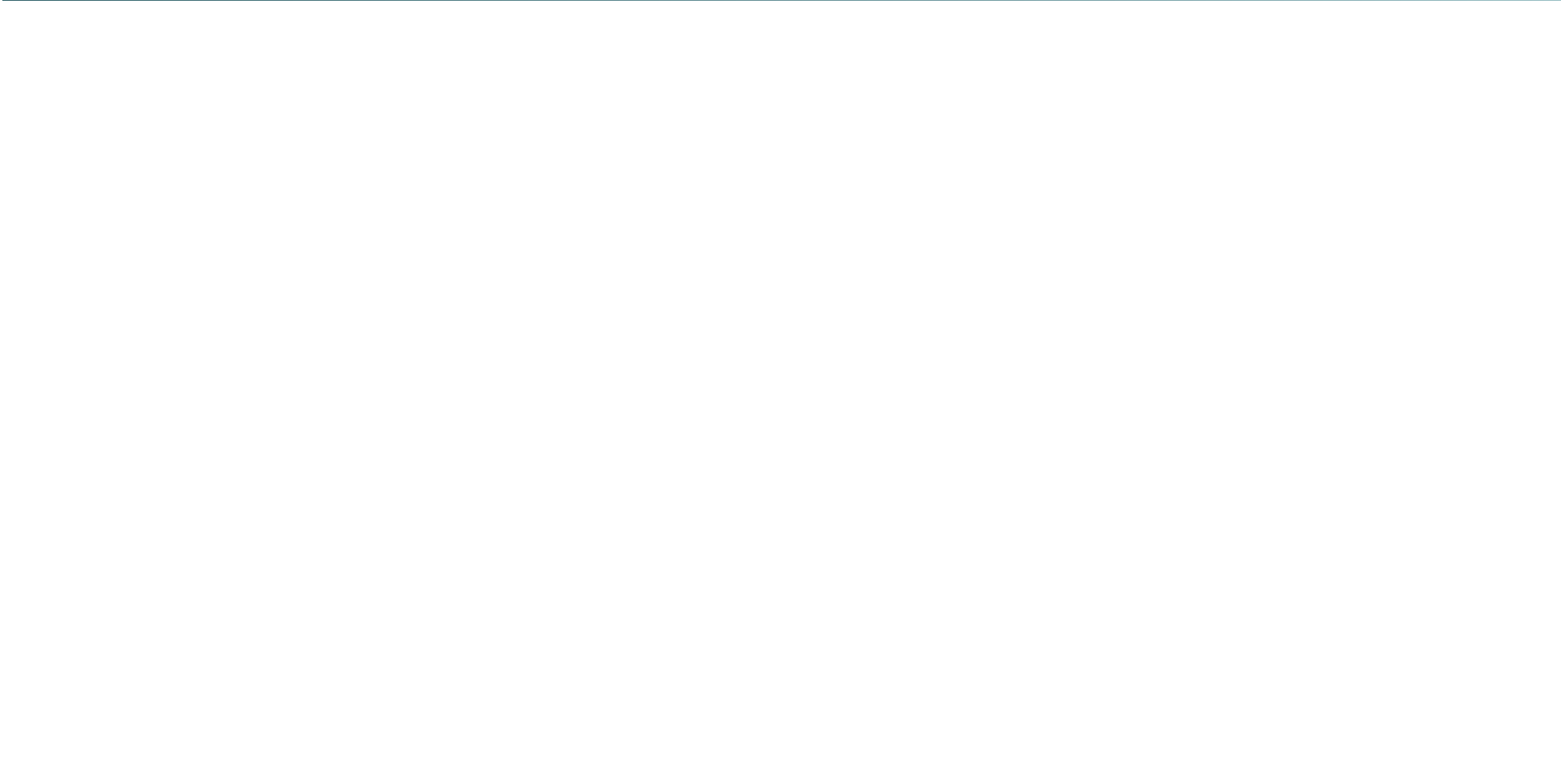
~~$(T_5 \cdot V_6 = 0.8)$~~
 $T_4 \cdot V_5 \approx 0.8$

Strengths

- Injects **private or real-time** knowledge not in pre-training
- **Reduces hallucination** by grounding answers in source documents
- **Citeable** – response can reference the source chunk
- No model weight update – knowledge base can be updated independently

Limitations

- Retrieval quality limits generation quality (garbage in, garbage out)
- Latency overhead from vector search at inference time
- Chunking strategy and embedding choice are non-trivial engineering decisions
- Does not change model reasoning or linguistic capabilities





Fine-Tuning: Definition and Variants

What Is Fine-Tuning?

Fine-tuning is continued training of a pre-trained foundation model on a curated, task-specific dataset, resulting in **updated model weights** that encode the new behaviour. Unlike prompting and RAG, fine-tuning permanently modifies the model checkpoint.



Fine-Tuning: Definition and Variants

Wavv

Main Fine-Tuning Variants

- **Instruction Fine-Tuning (IFT):** Train on (instruction, response) pairs to improve instruction following
- **Full Fine-Tuning:** All parameters updated; highest performance gain; highest compute cost
- **LoRA (Low-Rank Adaptation):** Freeze base weights; add low-rank adapter matrices; <1% parameters trained
- **QLoRA:** LoRA applied to a 4-bit quantised base model; enables fine-tuning on consumer GPUs
- **RLHF:** Reinforcement Learning from Human Feedback; aligns outputs with human preference



Fine-Tuning: Strengths and Limitations

Strengths

- Highest performance on narrow, well-defined tasks
- Persistent behaviour change – no prompt engineering required at inference
- Lower per-token inference cost vs. large system prompts
- Can instil format, tone, and domain vocabulary deeply

Limitations

- Requires labelled training data (hundreds to thousands of examples)
- Risk of **catastrophic forgetting**: erasing general capabilities
- Compute and storage cost for each new fine-tuned checkpoint
- Base model updates require re-fine-tuning from scratch

$r=1$
rank=1

Why LoRA Changed Fine-Tuning



The Core Idea

In a standard Transformer, each attention projection is a weight matrix $W \in \mathbb{R}^{d \times d}$. Full fine-tuning updates every element of W . LoRA instead **freezes** W and learns a low-rank decomposition:

$$\Delta W = A \cdot B, \quad A \in \mathbb{R}^{d \times r}, \quad B \in \mathbb{R}^{r \times d}, \quad r \ll d$$

Only A and B are trained – typically $r = 8$ to 64 , reducing trainable parameters by **99%** or more.

Practical Impact

Llama 3 8B has ~ 8 billion parameters. A LoRA adapter with $r=16$ adds ~ 20 million trainable parameters. This fits in a single 24 GB consumer GPU (e.g., RTX 4090) – making fine-tuning accessible without a data-centre budget.

Adaptation Strategy Comparison



Attribute	Prompt Engineering	RAG	Fine-Tuning
Weights up-dated?	No	No	Yes
Data required	None	Unstructured documents	Labelled (instruction, response) pairs
Setup time	Minutes	Hours to days	Days to weeks
Knowledge injection	Context window only	External corpus, real-time	Baked into weights
Hallucination risk	Moderate	Lower (grounded sources)	Low if data is clean
Typical use case	Task steering, persona	Enterprise Q&A, document search	Domain specialist, behaviour change
Example tools	Claude, GPT-4o system prompt	LangChain + Pinecone + GPT-4o	Llama 3 + LoRA + Axolotl



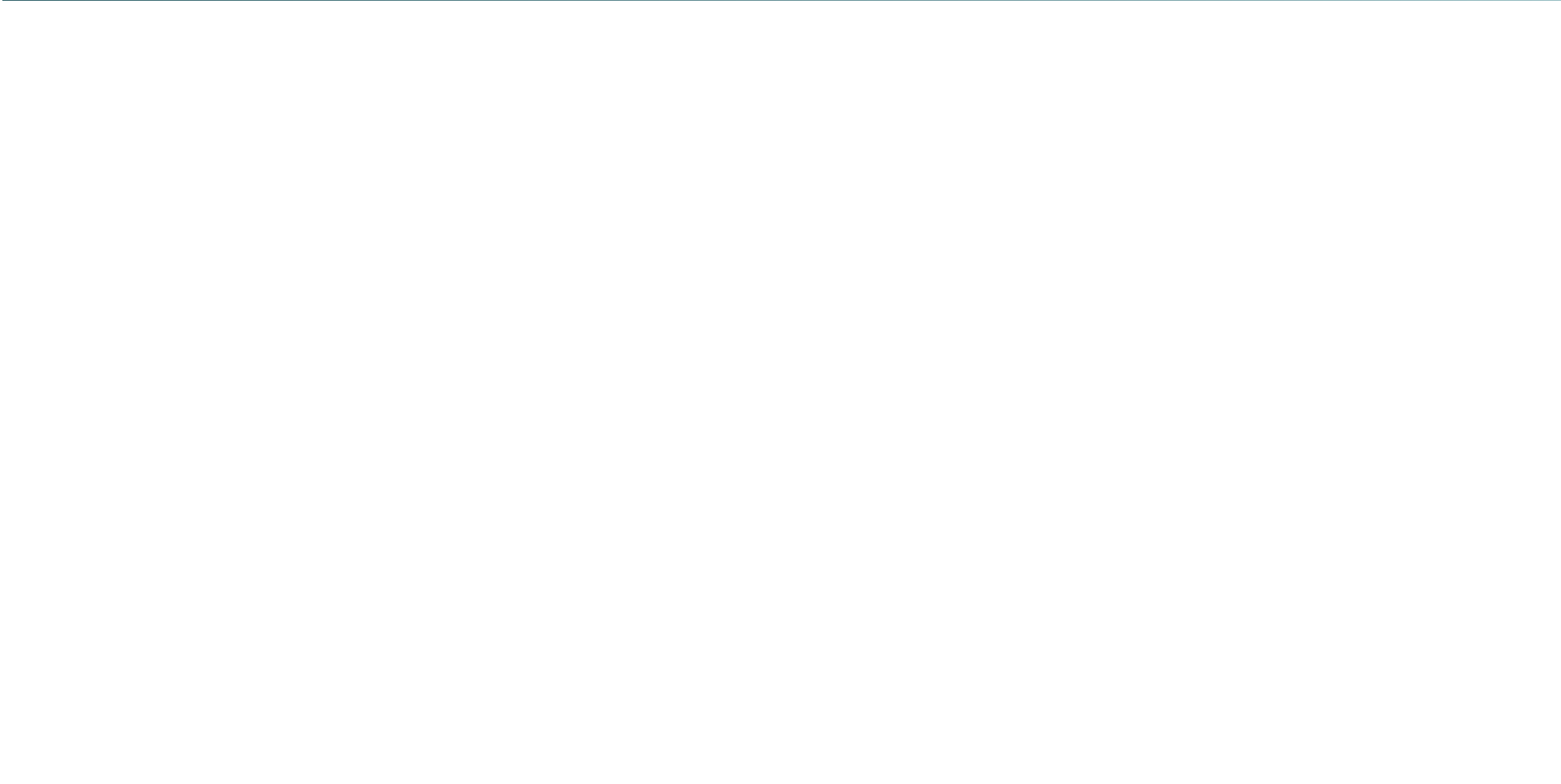
How to Choose: A Decision Framework

Ask These Questions in Order

1. **Can a clear instruction + examples solve it?** → Start with prompt engineering
2. **Does the model lack recent or private knowledge?** → Add RAG before fine-tuning
3. **Is task performance still insufficient, and do you have labelled data?** → Fine-tune
4. **Is GPU budget limited?** → Use LoRA or QLoRA over full fine-tuning

The Golden Rule

- **Always start with prompting.** It is the fastest feedback loop.
- **Add RAG** when grounding or freshness is needed.
- **Fine-tune last** – only when the above two strategies are demonstrably insufficient.
- The strategies are **composable**: RAG + fine-tuning is a common production stack.





Application: Intelligent Customer Support

Tier-1 Triage – Prompting

- System prompt defines product scope, tone, escalation rules
- Zero-shot handling of common questions (returns, shipping, billing)
- Works out of the box with GPT-4o or Claude 3.5
- Deployed in <1 day; iterates in real time

Knowledge-Grounded – RAG

- Product manuals, FAQs, and policy docs indexed in Pinecone
- Retrieves relevant clauses before generating reply
- Handles long-tail questions not coverable by prompts alone
- Freshness: knowledge base updated without model retraining



Application: Domain-Expert AI (Legal & Medical)

Why Fine-Tuning Dominates Here

General-purpose LLMs produce fluent text in legal and medical domains but lack **jurisdiction-specific terminology**, **clinical reasoning patterns**, and strict **format compliance**. Fine-tuning on expert-curated corpora closes this gap systematically and persistently, without relying on long system prompts that can be overridden.

Production Examples

- **Harvey AI:** GPT-4 base fine-tuned on legal briefs and case law for contract drafting
- **Med-PaLM 2 (Google):** Fine-tuned on clinical notes and medical board exam QA pairs
- **ClinicalBERT:** BERT fine-tuned on MIMIC-III clinical notes for clinical NLP tasks
- **BioMedLM (Stanford):** GPT-2 XL fine-tuned on biomedical literature



Application: Code Intelligence Platforms

Strategy: Prompting + RAG

- System prompt sets language, style guide, and code conventions
- RAG retrieves relevant internal codebase functions and docstrings
- Cursor IDE: repo-level context passed at inference via RAG
- Handles open-source libraries without fine-tuning

Strategy: Fine-Tuning

- GitHub Copilot: fine-tuned on billions of code tokens (public repos)
- Code Llama: Llama 2 fine-tuned specifically on code corpora (Meta)
- StarCoder 2: fine-tuned on The Stack v2 (600+ programming languages)
- Higher autocomplete accuracy vs. prompting alone on low-resource languages



Application: Enterprise Knowledge Management

The Classic RAG Use Case

Large enterprises generate enormous volumes of internal documents: policy manuals, engineering runbooks, HR guides, contracts, meeting transcripts. Employees spend hours locating information. RAG converts this corpus into a **conversational knowledge base** grounded in authoritative internal sources, with citations and freshness guarantees that fine-tuning cannot provide.

Production Stack

- **Embedding:** text-embedding-3-large (OpenAI) or E5-large-v2 (open-weight)
- **Vector store:** Azure AI Search, Pinecone, or pgvector in Postgres
- **Orchestration:** LlamaIndex or LangChain with hybrid search (BM25 + dense)
- **Generator:** GPT-4o or Claude 3.5 Sonnet with source-citation instructions



Knowledge Graph: Where This Concept Lives

Prerequisite Concepts

- Level 1: Foundation Models (previous concept)
- Generative vs. Discriminative Models (Module 1)
- What is a Language Model? (Module 1)
- Five-Level GenAI Framework (Module 1)

Concepts This Unlocks

- Prompt Engineering – zero to few-shot (Module 2)
- Chain-of-Thought and Advanced Prompting (Module 2)
- RAG Architecture Overview (Module 5)
- What is Fine-Tuning? (Module 7)
- LoRA and QLoRA in Practice (Module 7)



Common Misconceptions

Critical Corrections

- **"RAG is fine-tuning the retriever"** – RAG retrieves at inference time; the embedding model and LLM weights are not updated during normal RAG operation
- **"Fine-tuning always outperforms prompting"** – On many tasks a well-crafted few-shot prompt with GPT-4o matches or exceeds a fine-tuned smaller model
- **"LoRA destroys the base model"** – LoRA adapters are additive; the frozen base weights are unchanged and the adapter can be detached at any time
- **"RAG prevents hallucinations completely"** – Retrieval can return irrelevant chunks; the model may still hallucinate when retrieved context is insufficient or misleading
- **"RLHF is just fine-tuning with better data"** – RLHF uses a learned reward model and policy gradient optimisation (PPO), which is architecturally distinct from supervised fine-tuning

Session Summary



Key Takeaways

- Three strategies adapt foundation models: prompting, RAG, and fine-tuning
- They differ fundamentally in cost, data requirements, and persistence
- Always start with prompting; add RAG for knowledge; fine-tune for behaviour
- Strategies are composable – RAG + fine-tuning is common in production

What Comes Next

- **Next concept:** Prompt Engineering deep-dive – zero-shot, few-shot, CoT, tree-of-thought
- **Later:** RAG architecture (Module 5), LoRA / QLoRA hands-on (Module 7)
- **Lab:** Implement a minimal RAG pipeline using LlamaIndex and a local Llama 3 model
- These three strategies will appear in every module from here to the Capstone

Thank You

Module 2: Generative AI – Core Concepts & Architecture
Level 2: Model Adaptation · Prof. Sashikumaar Ganesan · IISc
Bengaluru